

Contents

- Start measuring time
- Read the image
- Image information
- Find frequency of each gray level
- Calculate probability
- Create Huffman dictionary
- Display Huffman coding table
- Calculate code lengths
- Average Code Length
- Entropy
- Calculate compressed size
- Original image size
- Bits saved
- Compression ratio
- Compression percentage
- Coding efficiency
- Measure computational time
- CPU time used
- Display final results

```
clc;  
clear;  
close all;
```

Start measuring time

```
startCPU = cputime;  
tic;
```

Read the image

```
img = imread('wheat.jpg');  
  
% Convert RGB image to grayscale  
if size(img, 3) == 3  
    img = rgb2gray(img);  
end
```

Image information

```
[rows, cols] = size(img);
```

```
totalPixels = numel(img);
fprintf('\n          HUFFMAN IMAGE COMPRESSION\n');

fprintf('\nImage Information:\n');
fprintf('Image Size      : %d x %d pixels\n', rows, cols);
fprintf('Total Pixels      : %d\n', totalPixels);
fprintf('Bits per Pixel     : 8\n');
fprintf('Original Size      : %d bits\n', totalPixels * 8);
```

HUFFMAN IMAGE COMPRESSION

```
Image Information:
Image Size      : 914 x 1280 pixels
Total Pixels    : 1169920
Bits per Pixel  : 8
Original Size   : 9359360 bits
```

Find frequency of each gray level

```
[counts, grayLevels] = imhist(img);

% Remove gray levels which do not occur
valid = counts > 0;

symbols = grayLevels(valid);
frequency = counts(valid);
```

Calculate probability

```
probability = frequency / totalPixels;
```

Create Huffman dictionary

```
huffmanDict = huffmandict(symbols, probability);
```

Display Huffman coding table

```
fprintf('\nGray Level | Frequency | Probability | Code\n');

for i = 1:length(symbols)

    code = huffmanDict{i, 2};

    % Convert code into readable binary form
    codeString = sprintf('%d', code);

    fprintf('%10d  %9d  %11.5f  %s\n', ...
        symbols(i), frequency(i), probability(i), codeString);

end
```

Gray Level	Frequency	Probability	Code
0	1384	0.00118	0011110001
1	5068	0.00433	01010000
2	4449	0.00380	01110100
3	3585	0.00306	11011000
4	3702	0.00316	10110101
5	3147	0.00269	000110000
6	2804	0.00240	001101001
7	2957	0.00253	001010000
8	2433	0.00208	010110101
9	2360	0.00202	011000101
10	2480	0.00212	010101011
11	2299	0.00197	011001010
12	2361	0.00202	011000100
13	2630	0.00225	010001101
14	2290	0.00196	011010101
15	2271	0.00194	011011011
16	2412	0.00206	010110110
17	2156	0.00184	011101111
18	2070	0.00177	100001101
19	1935	0.00165	100111110
20	1765	0.00151	111001010
21	1829	0.00156	101111100
22	1648	0.00141	0000101000
23	1498	0.00128	0010010101
24	1459	0.00125	0010101111
25	1350	0.00115	0011111100
26	1392	0.00119	0011010111
27	1325	0.00113	0100001111
28	1166	0.00100	0110010010
29	1346	0.00115	0011111101
30	1219	0.00104	0101101110
31	1250	0.00107	0101000111
32	1329	0.00114	0100001110
33	1254	0.00107	0101000110
34	1314	0.00112	0100100100
35	1366	0.00117	0011110110
36	1296	0.00111	0100100101
37	1450	0.00124	0011000010
38	1350	0.00115	0011110111
39	1391	0.00119	0011100110
40	1435	0.00123	0011000011
41	1385	0.00118	0011110000
42	1513	0.00129	0010010100
43	1471	0.00126	0010010111
44	1494	0.00128	0010010110
45	1550	0.00132	0001100011
46	1385	0.00118	0011100111
47	1709	0.00146	111110010
48	1593	0.00136	0001000110
49	1555	0.00133	0001000111
50	1688	0.00144	111110011
51	1613	0.00138	0000101001
52	1676	0.00143	0000000101
53	1677	0.00143	0000000100
54	1658	0.00142	00000100001

54	1696	0.00142	0000100001
55	1717	0.00147	111101001
56	1740	0.00149	111001011
57	1781	0.00152	110110011
58	1781	0.00152	110110010
59	1733	0.00148	111101000
60	1837	0.00157	101110010
61	1819	0.00155	101111110
62	1827	0.00156	101111101
63	1923	0.00164	100111111
64	1808	0.00155	110011101
65	1833	0.00157	101110011
66	1892	0.00162	101101000
67	1857	0.00159	101101001
68	2013	0.00172	100011001
69	1979	0.00169	100101000
70	1938	0.00166	100111101
71	2033	0.00174	100011000
72	1976	0.00169	100101001
73	2082	0.00178	100001100
74	2065	0.00177	100001110
75	2119	0.00181	100000010
76	2089	0.00179	100001011
77	2105	0.00180	100001010
78	2063	0.00176	100001111
79	2224	0.00190	011101010
80	2112	0.00181	100000011
81	2140	0.00183	100000001
82	2215	0.00189	011101110
83	2221	0.00190	011101011
84	2279	0.00195	011011010
85	2352	0.00201	011000110
86	2332	0.00199	011001000
87	2292	0.00196	011010100
88	2339	0.00200	011000111
89	2442	0.00209	010110100
90	2483	0.00212	010101010
91	2541	0.00217	010011110
92	2536	0.00217	010100010
93	2568	0.00220	010011101
94	2609	0.00223	010010011
95	2537	0.00217	010011111
96	2587	0.00221	010011100
97	2665	0.00228	010000110
98	2676	0.00229	001111111
99	2729	0.00233	001111001
100	2794	0.00239	001101010
101	2719	0.00232	001111010
102	2784	0.00238	001110010
103	2856	0.00244	001101000
104	2895	0.00247	001100000
105	2923	0.00250	001010110
106	2937	0.00251	001010001
107	3024	0.00258	001000101
108	3168	0.00271	000100010
109	3232	0.00276	000010101
110	3285	0.00281	000010001
111	3382	0.00289	000000000

---	----	-----	-----
112	3353	0.00287	000000001
113	3417	0.00292	11111000
114	3545	0.00303	11011011
115	3442	0.00294	11110101
116	3552	0.00304	11011010
117	3536	0.00302	11100100
118	3616	0.00309	11001111
119	3667	0.00313	10111101
120	3670	0.00314	10111100
121	3680	0.00315	10111000
122	3794	0.00324	10101010
123	3976	0.00340	10001101
124	3952	0.00338	10010101
125	4092	0.00350	10001000
126	4194	0.00358	10000100
127	4296	0.00367	01111111
128	4316	0.00369	01111110
129	4392	0.00375	01110110
130	4580	0.00391	01101011
131	4575	0.00391	01101100
132	4739	0.00405	01100001
133	4779	0.00408	01100000
134	4887	0.00418	01011001
135	5039	0.00431	01010100
136	5221	0.00446	01001000
137	5358	0.00458	01000010
138	5401	0.00462	00111110
139	5564	0.00476	00111000
140	5668	0.00484	00110001
141	6017	0.00514	00100011
142	5864	0.00501	00101001
143	6008	0.00514	00100100
144	6155	0.00526	00011001
145	6411	0.00548	00010000
146	6512	0.00557	00001001
147	6766	0.00578	1111101
148	6874	0.00588	1111011
149	6963	0.00595	1110011
150	7367	0.00630	1011011
151	7342	0.00628	1011101
152	7631	0.00652	1010100
153	7794	0.00666	1001110
154	7978	0.00682	1000111
155	8139	0.00696	1000101
156	8485	0.00725	1000001
157	8645	0.00739	0111110
158	8679	0.00742	0111100
159	8994	0.00769	0110111
160	8972	0.00767	0111000
161	8924	0.00763	0111001
162	9212	0.00787	0110100
163	9613	0.00822	0101111
164	9637	0.00824	0101110
165	9954	0.00851	0101011
166	10334	0.00883	0100110
167	10340	0.00884	0100101
168	10660	0.00911	0100010

169	10969	0.00938	0011101
170	11220	0.00959	0011011
171	11565	0.00989	0010110
172	11550	0.00987	0010111
173	11797	0.01008	0010011
174	12095	0.01034	0001111
175	12246	0.01047	0001110
176	12359	0.01056	0001101
177	12517	0.01070	0001010
178	12512	0.01069	0001011
179	12661	0.01082	0001001
180	12883	0.01101	0000110
181	13236	0.01131	0000010
182	13559	0.01159	111111
183	13396	0.01145	0000001
184	13769	0.01177	111100
185	13961	0.01193	111011
186	13983	0.01195	111010
187	14128	0.01208	110111
188	14388	0.01230	110101
189	14479	0.01238	110100
190	14597	0.01248	110001
191	14583	0.01246	110010
192	14976	0.01280	101100
193	15368	0.01314	101000
194	15633	0.01336	100110
195	15897	0.01359	100100
196	15236	0.01302	101001
197	15047	0.01286	101011
198	14622	0.01250	110000
199	14105	0.01206	111000
200	13178	0.01126	0000011
201	12787	0.01093	0000111
202	12094	0.01034	0010000
203	11366	0.00972	0011001
204	10705	0.00915	0100000
205	10047	0.00859	0101001
206	9226	0.00789	0110011
207	8676	0.00742	0111101
208	7803	0.00667	1001011
209	7252	0.00620	1100110
210	6464	0.00553	00001011
211	5852	0.00500	00101010
212	5247	0.00448	01000111
213	4895	0.00418	01011000
214	4085	0.00349	10001001
215	3787	0.00324	10101011
216	3344	0.00286	000000011
217	3047	0.00260	001000100
218	2644	0.00226	010001100
219	2297	0.00196	011001011
220	1952	0.00167	100111100
221	1814	0.00155	110011100
222	1550	0.00132	0001100010
223	1398	0.00119	0011010110
224	1176	0.00101	0101101111
225	1060	0.00091	1000000001

226	880	0.00075	00001000000
227	736	0.00063	00101011100
228	567	0.00048	01100100111
229	530	0.00045	10000000001
230	403	0.00034	000010000010
231	302	0.00026	011001001100
232	273	0.00023	100000000001
233	201	0.00017	101111111111
234	189	0.00016	0010101110110
235	142	0.00012	0110010011011
236	116	0.00010	1011111111100
237	97	0.00008	00001000001101
238	94	0.00008	00001000001111
239	97	0.00008	00001000001100
240	82	0.00007	00101011101110
241	72	0.00006	10000000000000
242	72	0.00006	00101011101111
243	69	0.00006	10000000000011
244	57	0.00005	10111111111010
245	69	0.00006	10000000000010
246	59	0.00005	101111111110001
247	69	0.00006	10000000000001
248	64	0.00005	101111111110000
249	96	0.00008	00001000001110
250	116	0.00010	10111111111001
251	146	0.00012	0110010011010
252	230	0.00020	1011111111101
253	383	0.00033	001010111010
254	931	0.00080	1011111110
255	44	0.00004	10111111111011

Calculate code lengths

```
codeLengths = zeros(length(symbols), 1);

for i = 1:length(symbols)
    codeLengths(i) = length(huffmanDict{i, 2});
end
```

Average Code Length

```
averageCodeLength = sum(probability .* codeLengths);
```

Entropy

```
entropyValue = -sum(probability .* log2(probability));
```

Calculate compressed size

```
compressedBits = sum(frequency .* codeLengths);
```

Original image size

```
originalBits = totalPixels * 8;
```

Bits saved

```
bitsSaved = originalBits - compressedBits;
```

Compression ratio

```
compressionRatio = originalBits / compressedBits;
```

Compression percentage

```
compressionPercentage = ...  
    (bitsSaved / originalBits) * 100;
```

Coding efficiency

```
efficiency = (entropyValue / averageCodeLength) * 100;
```

Measure computational time

```
executionTime = toc;
```

CPU time used

```
cpuTimeUsed = cputime - startCPU;
```

Display final results

```
fprintf('\n          COMPUTATIONAL RESULTS\n');  
  
fprintf('\nNumber of Symbols      : %d\n', length(symbols));  
  
fprintf('Entropy                  : %.5f bits/pixel\n', ...  
    entropyValue);  
  
fprintf('Average Code Length      : %.5f bits/pixel\n', ...  
    averageCodeLength);  
  
fprintf('Original Size            : %d bits\n', ...  
    originalBits);  
  
fprintf('Compressed Size          : %d bits\n', ...  
    compressedBits);  
  
fprintf('Bits Saved               : %d bits\n', ...  
    bitsSaved);
```

```

fprintf('Bits Saved           : %d bits\n', ...
        bitsSaved);

fprintf('Compression Ratio     : %.5f\n', ...
        compressionRatio);

fprintf('Compression Percentage : %.2f %%\n', ...
        compressionPercentage);

fprintf('Coding Efficiency      : %.2f %%\n', ...
        efficiency);

fprintf('\nExecution Time       : %.6f seconds\n', ...
        executionTime);

fprintf('CPU Time Used           : %.6f seconds\n', ...
        cpuTimeUsed);

```

COMPUTATIONAL RESULTS

```

Number of Symbols      : 256
Entropy                : 7.42171 bits/pixel
Average Code Length    : 7.45615 bits/pixel
Original Size          : 9359360 bits
Compressed Size        : 8723102 bits
Bits Saved             : 636258 bits
Compression Ratio      : 1.07294
Compression Percentage : 6.80 %
Coding Efficiency       : 99.54 %

Execution Time         : 0.160099 seconds
CPU Time Used          : 0.359375 seconds

```